

实践2：设计感知与动作空间，实现宇树G1粗糙地形行走策略



深蓝学院《人形机器人运动控制》课程第2章

课程链接：<https://www.shenlanxueyuan.com/course/864>

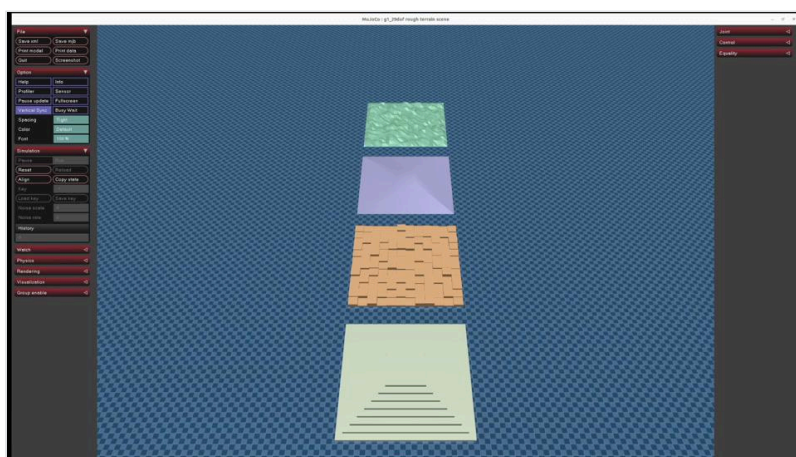
1. 作业目标

本次作业的目标是在 `unitree_rl_lab` 的基础上训练一个带地形感知的 Unitree G1 29DoF 粗糙地形行走策略，并通过 MuJoCo sim2sim 验证训练效果。

2. 作业背景

平地速度跟踪任务主要依赖本体感知，包括机身角速度、重力投影、速度命令、关节位置、关节速度和上一帧动作。机器人在粗糙地形上行走时，仅依赖本体感知通常不够，因为策略无法提前知道脚下地形高度变化。

本次作业要求在平地行走任务的基础上增加一个类似 IsaacLab `height_scanner` 的地形高度扫描观测。策略需要同时利用本体状态、速度命令、上一帧动作和局部地形高度信息，从而学习更适合粗糙地形的运动控制策略。



3. 环境准备

3.1 安装 IsaacLab 和 unitree_rl_lab



如果在第1章实践1中，已经安装好了环境，则可略过。

环境配置建议：IsaacSim 5.1.0, IsaacLab v2.3.2, conda 默认环境名称为 env_isaialab

完成本章的实践2，需先安装 IsaacLab，然后从 GitHub 下载 `unitree_rl_lab`，并按照仓库 README 完成安装。参考仓库：https://github.com/unitreerobotics/unitree_rl_lab

3.2 作业代码目录

本次作业的目录结构：

代码块

```
1  unitree_rl_lab/
2  |
3  |-- source/unitree_rl_lab/
4  |   |-- unitree_rl_lab/
5  |       |-- tasks/
6  |           |-- locomotion/
7  |               |-- robots/
8  |                   |-- g1/29dof/
9  |                       |-- velocity_env_cfg.py
10 |                           |-- rough_env_cfg.py #本次作业需要完成的粗糙地形感知任务
11 |                               |-- __init__.py
12 |
13 |-- sim2sim/ #课程代码ch2_sim2sim.zip提供
14 |   |-- assets/
15 |       |-- g1_29dof_raycaster.xml
16 |       |-- scene_rough.xml
17 |       |-- scene_flat.xml
18 |       |-- rough_terrain_generator.py
19 |       |-- meshes/
20 |   |-- config.py
21 |   |-- mujoco_env.py
22 |   |-- policy_inference.py
23 |   |-- sim2sim_raycaster.py
```

其中：

- `velocity_env_cfg.py`：G1 29DoF 平地或基础速度任务配置，可作为粗糙地形任务的参考。
- `rough_env_cfg.py`：本次作业需要完成的粗糙地形感知任务配置，包含 `ROUGH_TERRAINS_CFG`、`height_scanner` 和 policy 观测配置。
- `__init__.py`：任务注册入口，需要注册 `Unitree-G1-29dof-Velocity-Rough`。
- `sim2sim/config.py`：MuJoCo sim2sim 的路径、policy、deploy 配置、仿真参数。

- `sim2sim/mujoco_env.py` : MuJoCo 环境封装, 负责加载模型、raycaster、高度扫描。
- `sim2sim/policy_inference.py` : 加载导出的 policy 并执行推理。
- `sim2sim/sim2sim_raycaster.py` : sim2sim 主程序, 负责读取观测、拼接历史帧、运行 policy、PD 控制和键盘命令。
- `sim2sim/assets/g1_29dof_raycaster.xml` : 带 raycaster camera 的 G1 29DoF 机器人模型。
- `sim2sim/assets/scene_rough.xml` : 粗糙地形 MuJoCo 场景, 包含台阶、box、斜坡、随机粗糙地形和 raycaster 插件配置。
- `sim2sim/assets/scene_flat.xml` : 平地快速测试场景。
- `sim2sim/assets/rough_terrain_generator.py` : 粗糙地形 XML 生成脚本。

4. 作业核心任务

- 任务 ID: `Unitree-G1-29dof-Velocity-Rough`
- 任务配置文件: `rough_env_cfg.py`
- 参考配置文件: `velocity_env_cfg.py`
- 地形配置: 必须使用 `ROUGH_TERRAINS_CFG`
- 感知观测: 必须把 `height_scan` 加入 policy 观测
- 验证方式: 训练完成后使用 `unitree_rl_lab/sim2sim` 进行键盘控制 sim2sim 测试

5. 作业提示

5.1 创建 `rough_env_cfg.py`

建议先把 `velocity_env_cfg.py` 复制一份, 并重命名为 `rough_env_cfg.py`, 这样可以保留 G1 29DoF 平地速度任务中已有的机器人、动作空间、奖励和训练参数, 再逐步改成粗糙地形感知任务。在 `rough_env_cfg.py` 中类名建议从原来的平地任务类名改成 `G1RoughEnvCfg` 和 `G1RoughPlayEnvCfg`。

5.2 注册任务 ID

任务注册为:

代码块

```
1 Unitree-G1-29dof-Velocity-Rough
```

注册位置为:

```
code
source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/robots/g1/29dof/__init__.py
```

注册时可以模仿已有的 `Unitree-G1-29dof-Velocity` 任务配置：先复制原来的注册项，再把任务 ID、环境配置入口和实验名改成粗糙地形任务对应的内容。新的任务应加载

`rough_env_cfg.py` 中的环境配置，例如 `G1RoughEnvCfg` 和 `G1RoughPlayEnvCfg`，并修改id为 `unitree_g1_29dof_velocity_rough`。

5.3 配置粗糙地形

`rough_env_cfg.py` 需要从 IsaacLab 引入粗糙地形配置：

代码块

```
1 from isaacsim.terrains.config.rough import ROUGH_TERRAINS_CFG
```

场景中的 terrain 必须使用 generator 类型，并设置为 `ROUGH_TERRAINS_CFG`：

代码块

```
1 terrain = TerrainImporterCfg(
2     prim_path="/World/ground",
3     terrain_type="generator",
4     terrain_generator=ROUGH_TERRAINS_CFG,
5     max_init_terrain_level=ROUGH_TERRAINS_CFG.num_rows - 1,
6     collision_group=-1,
7 )
```

5.4 添加 height_scanner 传感器

在 `rough_env_cfg.py` 的 scene 配置中添加传感器 `RayCasterCfg`。推荐配置如下：

代码块

```
1 height_scanner = RayCasterCfg(
2     prim_path="{ENV_REGEX_NS}/Robot/torso_link",
3     offset=RayCasterCfg.OffsetCfg(pos=(0.0, 0.0, 20.0)),
4     ray_alignment="yaw",
5     pattern_cfg=patterns.GridPatternCfg(resolution=0.1, size=[1.6, 1.0]),
6     debug_vis=True,
7     mesh_prim_paths=["/World/ground"],
8 )
```

配置含义：

- `prim_path="{ENV_REGEX_NS}/Robot/torso_link"`：扫描器跟随机器人躯干位置。
- `offset=(0.0, 0.0, 20.0)`：从机器人上方往地面发射射线。
- `ray_alignment="yaw"`：扫描网格只跟随机器人 yaw 方向，忽略 roll/pitch 晃动。
- `resolution=0.1`：网格间距为 0.1 m。
- `size=[1.6, 1.0]`：扫描区域前后 1.6 m、左右 1.0 m。
- `mesh_prim_paths=["/World/ground"]`：只扫描地形，不扫描机器人身体。

按照该配置，height scanner 的点数为：

代码块

```
1 (1.6 / 0.1 + 1) x (1.0 / 0.1 + 1) = 17 x 11 = 187
```

5.5 添加 height_scan 观测项

在 `ObservationsCfg.PolicyCfg` 中加入：

代码块

```
1 height_scanner = ObsTerm(  
2     func=mdp.height_scan,  
3     params={"sensor_cfg": SceneEntityCfg("height_scanner")},  
4     clip=(-1.0, 5.0),  
5 )
```

也建议在 `ObservationsCfg.CriticCfg` 中加入同样的观测项，让 critic 在训练时也能看到地形信息。

IsaacLab 的 `height_scan` 观测函数定义方式如下：

代码块

```
1 def height_scan(env: ManagerBasedEnv, sensor_cfg: SceneEntityCfg, offset:  
2     float = 0.5) -> torch.Tensor:  
3     """Height scan from the given sensor w.r.t. the sensor's frame.  
4     The provided offset (Defaults to 0.5) is subtracted from the returned  
5     values.  
6     # extract the used quantities (to enable type-hinting)  
7     sensor: RayCaster = env.scene.sensors[sensor_cfg.name]  
8     # height scan: height = sensor_height - hit_point_z - offset
```

```
9     return sensor.data.pos_w[:, 2].unsqueeze(1) - sensor.data.ray_hits_w[:, 2] - offset
```

`height_scan` 的返回值不是直接输出地形的绝对高度，而是输出“传感器高度减去命中点高度”后的相对高度差。`sensor.data.pos_w[:, 2]` 表示传感器原点在世界坐标系中的 z 轴高度，`sensor.data.ray_hits_w[:, 2]` 表示每条射线命中点的世界坐标 z 值，两者相减得到的是传感器到地面的竖直距离，再减去 `offset` 后，就把高度图的零点对齐到了训练时约定的参考平面上。

`offset=0.5` 是 IsaacLab 里 `height_scan` 的默认偏移量。它不是从机器人几何严格推导出来的物理常数，而是训练时为了统一观测分布而采用的零点定义。也就是说，当命中点刚好落在这个参考平面上时，输出会接近零；地形比参考平面低时输出为正，比参考平面高时输出为负。部署时如果要复现训练时的 policy 输入，就必须保持这个偏移一致。

添加 `height_scan` 观测项后，policy 观测如下：

代码块

```
1  base_ang_vel
2  projected_gravity
3  velocity_commands
4  joint_pos_rel
5  joint_vel_rel
6  last_action
7  height_scanner
```

单帧观测维度为：

代码块

```
1  3 + 3 + 3 + 29 + 29 + 29 + 187 = 283
```

如果保留 `history_length = 5`，最终 policy 输入维度为：

代码块

```
1  283 x 5 = 1415
```

如果你在配置中额外加入了其他观测项，最终维度可以不同，但必须在报告中说明原因。最终维度应以你自己的 `rough_env_cfg.py` 和训练 run 中导出的 `deploy.yaml` 为准。

5.6 动作空间要求

本次作业的动作空间保持为 G1 29DoF 关节位置控制，不要求修改动作维度：

代码块

```
1 JointPositionAction = mdp.JointPositionActionCfg(  
2     asset_name="robot",  
3     joint_names=[".*"],  
4     scale=0.25,  
5     use_default_offset=True,  
6 )
```

5.7 修改奖励函数配置和权重

`rough_env_cfg.py` 可以先保留 `velocity_env_cfg.py` 中的大部分奖励项，但需要根据粗糙地形任务适当检查和调整奖励权重，例如速度跟踪、身体高度等相关奖励。例如：在 `base_height` 中传递 `sensor_cfg` 参数，这样会变成追踪机器人相对地面的高度，而不是世界坐标系的绝对高度，想一想为什么需要这么做？

代码块

```
1 base_height = RewTerm(  
2     func=mdp.base_height_l2,  
3     weight=-10,  
4     params={  
5         "target_height": 0.78,  
6         "sensor_cfg": SceneEntityCfg("height_scanner")  
7     }  
8 )
```

还有 `foot_clearance_reward` 也是针对平地行走任务的奖励，`target_height` 是世界坐标系的绝对高度，应该修改为机器人（脚）相对地面的高度。

为让机器人学会正常行走，可添加 `feet_air_time` 奖励函数，该函数奖励双足机器人在规定阈值时间内迈步，同时保持每次只有一只脚在空中，防止机器人双脚同时腾空，出现跳跃式步态，保证稳定行走。

代码块

```
1 feet_air_time = RewTerm(  
2     func=mdp.feet_air_time_positive_biped,  
3     weight=0.5,  
4     params={  
5         "command_name": "base_velocity",  
6         "sensor_cfg": SceneEntityCfg("contact_forces",  
7         body_names=".*_ankle_roll_link"),  
8         "threshold": 0.5,
```

```
8         },
9     )
```

5.8 修改 termination 配置

粗糙地形任务中也需要检查 `TerminationsCfg`。平地任务里的终止条件可能不全部适用于粗糙地形，例如 `base height` 条件，这一项的含义是：当机器人的 root（基座/躯干）在世界坐标系中的绝对高度低于 0.2 米时，episode 终止。在复杂地形下不建议开启该条件，因为机器人初始位置可能就在低洼处，root 高度可能一上来就低于 0.2m，直接终止导致训练无法正常进行。

如果终止条件中只保留 `time_out` 和 `bad_orientation`，机器人仍然可能陷入局部最优解，例如学到坐在地上或者用非足端部位支撑身体策略。虽然这类行为不一定会触发姿态异常终止，但显然不是期望的运动模式。因此，仍然需要加入非法接触相关的终止项，用来约束机器人避免用膝盖、躯干、手臂等非预期部位与地形发生持续接触。

在 `unitree_rl_lab` 中如果直接使用 `IsaacLab` 的 `illegal_contact = DoneTerm(func=mdp.illegal_contact, ...)`，可能会因为 reset 初期接触力瞬间达到阈值，导致机器人被不停重置，训练无法正常展开。

一种处理方式是吧 `illegal_contact` 改成 `illegal_reset_contact`，让非法接触连续或累计多次达到阈值后再触发 reset。

在 `rough_env_cfg.py` 的 `TerminationsCfg` 中使用：

代码块

```
1  # illegal_contact = DoneTerm(
2  #     func=mdp.illegal_contact,
3  #     time_out=False,
4  #     params={"sensor_cfg": SceneEntityCfg("contact_forces", body_names=
5  #         ["torso_link"]), "threshold": 1.0},
6  # )
7  illegal_reset_contact = DoneTerm(
8      func=mdp.illegal_reset_contact,
9      time_out=True,
10     params={
11         "sensor_cfg": SceneEntityCfg("contact_forces", body_names=
12             ["torso_link"]),
13         "threshold": 1.0,
14         "episode_length_threshold": 5,
15         "print_reason": False,
16     },
17 )
```

然后在下面路径新增或修改 termination 实现：

代码块

```
1 source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/mdp/terminations.py
```

实现示例：

代码块

```
1  from __future__ import annotations
2
3  from typing import TYPE_CHECKING, Sequence
4
5  import torch
6
7  from isaacsim.managers import ManagerTermBase, ManagerTermBaseCfg,
   SceneEntityCfg
8  from isaacsim.sensors import ContactSensor
9
10 if TYPE_CHECKING:
11     from isaacsim.envs import ManagerBasedRLEnv
12
13
14 class illegal_reset_contact(ManagerTermBase):
15     def __init__(self, cfg: ManagerTermBaseCfg, env: ManagerBasedRLEnv):
16         super().__init__(cfg, env)
17         self.threshold = cfg.params["threshold"]
18         self.sensor_cfg = cfg.params["sensor_cfg"]
19         self.print_reason = cfg.params.get("print_reason", False)
20         self.episode_length_threshold =
   cfg.params.get("episode_length_threshold", 1)
21         self.illegal_contact_counter = torch.zeros(env.num_envs,
   device=env.device, dtype=torch.int)
22
23     def __call__(
24         self,
25         env: ManagerBasedRLEnv,
26         threshold: float,
27         sensor_cfg: SceneEntityCfg,
28         print_reason: bool = False,
29         episode_length_threshold: int = 1,
30     ) -> torch.Tensor:
31         contact_sensor: ContactSensor = env.scene.sensors[sensor_cfg.name]
32         net_contact_forces = contact_sensor.data.net_forces_w_history
33         contacts = torch.any(
34             torch.max(torch.norm(net_contact_forces[:, :,
   sensor_cfg.body_ids], dim=-1), dim=1)[0] > threshold,
```

```

35         dim=1,
36     )
37     self.illegal_contact_counter += contacts.int()
38     # if contact count and episode length exceed the first
episode_length_threshold steps, trigger termination
39     reset_envs = torch.logical_and(
40         self.illegal_contact_counter >= episode_length_threshold,
41         env.episode_length_buf >= episode_length_threshold,
42     )
43
44     if reset_envs.any() and print_reason:
45         print(f"illegal_reset_contact: {reset_envs.sum()} envs")
46
47     return reset_envs
48
49     def reset(self, env_ids: Sequence[int] | slice | None = None) -> None:
50         if env_ids is None:
51             env_ids = slice(None)
52         self.illegal_contact_counter[env_ids] = 0

```

然后确认下面文件中已经导入自定义 termination:

代码块

```
1 source/unitree_rl_lab/unitree_rl_lab/tasks/locomotion/mdp/__init__.py
```

需要包含:

代码块

```
1 from .terminations import * # noqa: F401, F403
```

6. 训练方法

训练命令如下:

代码块

```

1 cd /path/to/unitree_rl_lab
2 ./unitree_rl_lab.sh -t \
3     --task Unitree-G1-29dof-Velocity-Rough \
4     --experiment_name unitree_g1_29dof_velocity_rough \
5     --max_iterations 15000 \
6     --num_envs 4096 \

```

```
7 --headless
```

如果显存不足，可以先降低并行环境数量：

代码块

```
1 ./unitree_rl_lab.sh -t \  
2 --task Unitree-G1-29dof-Velocity-Rough \  
3 --experiment_name unitree_g1_29dof_velocity_rough \  
4 --max_iterations 15000 \  
5 --num_envs 1024 \  
6 --headless
```

训练日志通常保存到：

代码块

```
1 logs/rsl_rl/unitree_g1_29dof_velocity_rough/<日期时间>/
```

查看训练曲线：

代码块

```
1 tensorboard --logdir ./logs/rsl_rl/
```

训练完成后，需要至少保存一个可用于验证的 checkpoint，例如：

代码块

```
1 model_14000.pt
```

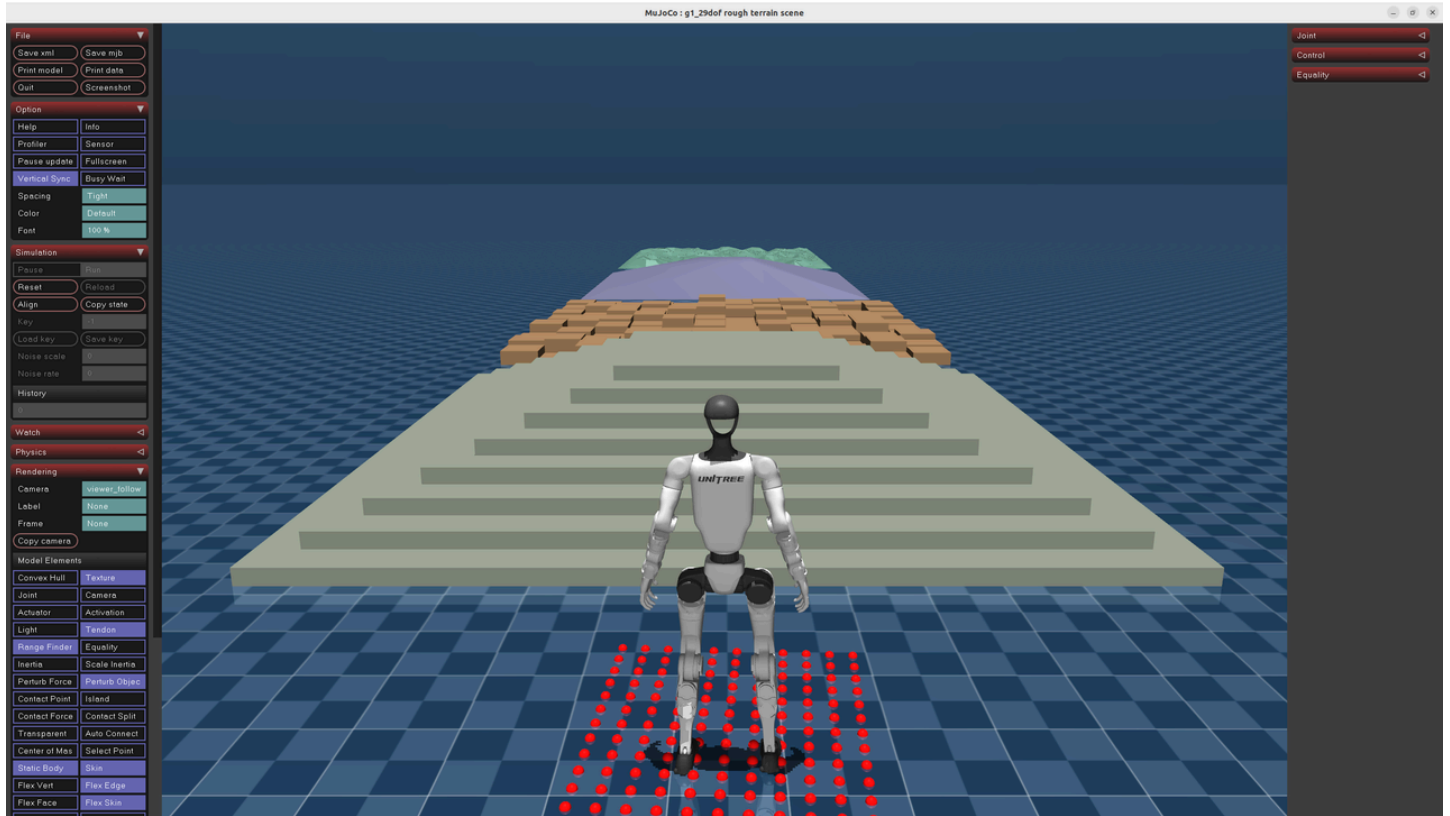
可以使用 play 命令在 IsaacLab 中先检查策略效果：

代码块

```
1 ./unitree_rl_lab.sh -p \  
2 --task Unitree-G1-29dof-Velocity-Rough \  
3 --experiment_name unitree_g1_29dof_velocity_rough \  
4 --num_envs 64 \  
5 --checkpoint /path/to/model_xxxx.pt
```

7. Sim2Sim 验证环境配置

请先将课程附件 `ch2_sim2sim.zip` 解压到 `unitree_rl_lab` 目录下面。训练完成后，可以使用 `unitree_rl_lab/sim2sim` 进行 MuJoCo sim2sim 验证。本项目提供的 sim2sim 不走 DDS 通信，而是直接在 Python 中加载 MuJoCo、读取 raycaster `sensordata`、拼接观测并运行 policy。视频演示链接：<https://www.bilibili.com/video/BV1n4jw6nExJ/>



7.1 下载并编译 MuJoCo raycaster 插件

sim2sim 中的 height scanner 依赖 MuJoCo raycaster 插件：

- 插件仓库：https://github.com/Albusgive/mujoco_ray_caster.git
- 编译步骤：参考插件仓库 README。

编译成功后，需要找到生成的动态库路径，例

如：`/path/to/mujoco/build/lib/libsensor_raycaster.so`

后续需要把这个实际路径填入 `sim2sim/config.py` 的 `RAYCASTER_PLUGIN_LIBRARY`。

需要注意：编译插件时使用的 MuJoCo 版本应和 Python 运行时加载的 MuJoCo 版本一致，否则可能出现 `plugin mujoco.sensor.ray_caster not found` 或 ABI 不兼容问题。

7.2 在训练环境中安装 MuJoCo Python 包

在训练使用的 conda 环境中安装 `mujoco`。安装版本应和 raycaster 插件编译时使用的 MuJoCo 版本一致。

可以先通过插件动态库查询其链接的 MuJoCo 版本：

```
code /path/to/mujoco/build/lib/libsensor_raycaster.so
```

如果输出中出现类似: `libmujoco.so.<插件对应的 MuJoCo 版本>`

则在训练环境中安装对应版本:

代码块

```
1 conda activate env_isaacrlab
2 python -m pip install mujoco==<插件对应的 MuJoCo 版本>
```

安装后可以验证 Python 包版本:

代码块

```
1 python -c "import mujoco; print(mujoco.__version__)"
```

7.3 配置 sim2sim/config.py

打开:

代码块

```
1 /path/to/unitree_rl_lab/sim2sim/config.py
```

至少需要检查以下几项:

代码块

```
1 SIM2SIM_DIR = "/path/to/unitree_rl_lab/sim2sim"
2 ASSETS_DIR = os.path.join(SIM2SIM_DIR, "assets")
3
4 TRAIN_RUN_DIR = (
5     "/path/to/unitree_rl_lab/logs/rsl_rl/"
6     "unitree_g1_29dof_velocity_rough/<run_time>"
7 )
8
9 ROBOT_SCENE = os.path.join(ASSETS_DIR, "scene_rough.xml")
10
11 RAYCASTER_PLUGIN_LIBRARY = "/path/to/mujoco/build/lib/libsensor_raycaster.so"
12
13 POLICY_PATH = os.path.join(TRAIN_RUN_DIR, "model_14000.pt")
14 DEPLOY_CONFIG = os.path.join(TRAIN_RUN_DIR, "params", "deploy.yaml")
```

其中最重要的是：

- `RAYCASTER_PLUGIN_LIBRARY` 指向 `libsensor_raycaster.so` 的路径。
- `ROBOT_SCENE` 指向 `sim2sim/assets/` 下的 `scene_rough.xml`。
- `TRAIN_RUN_DIR` 指向本次粗糙地形训练产生的日志目录。
- `POLICY_PATH` 指向 `TRAIN_RUN_DIR` 的 `checkpoint`。
- `DEPLOY_CONFIG` 指向 `TRAIN_RUN_DIR` 的 `params/deploy.yaml`。

如果路径不对，常见报错包括：

- `plugin mujoco.sensor.ray_caster not found`
- `Error opening file scene_rough.xml`
- `No such file or directory: model_xxxx.pt`
- 观测维度和策略输入维度不一致

7.4 运行 sim2sim

启动可视化键盘控制：

代码块

```
1  conda activate env_isaacLab
2  cd /path/to/unitree_rl_lab/sim2sim
3  python sim2sim_raycaster.py
```

键盘控制：

- 方向键 `↑/↓` 或 小键盘 `8/2`：前进/后退速度
- 方向键 `←/→` 或 小键盘 `4/6`：左右转向角速度
- `Space` 或 小键盘 `5`：停止
- `R`：重置仿真

验证时需要观察：

- 红色 raycaster 点是否落在地形上，而不是机器人身体上。
- 机器人能否在粗糙地形上保持站立并响应速度命令。
- policy 输入维度是否和训练时一致。
- `height_scan` 的点数、顺序、裁剪范围是否和 IsaacLab 配置一致。

8. 提交内容

每位同学需要提交以下内容：

1. 修改后的 `rough_env_cfg.py`。
2. 至少一个训练 checkpoint，例如 `model_14000.pt` 或 `policy.pt`。
3. isaacsim 或者 mujoco sim2sim 运行视频，建议视频长度不超过2分钟。
4. 一份简短报告，说明你从平地任务到粗糙地形任务具体改了哪些观测、传感器、地形和奖励配置。

9. 评分标准

总分 100 分：

项目	分值	要求
<code>rough_env_cfg.py</code> 等配置代码（只提交你修改过的程序即可，不需要提交未经修改的unitree_rl_lab原始代码）	40	正确使用 <code>ROUGH_TERRAINS_CFG</code> ；正确添加 <code>height_scanner = RayCasterCfg(...)</code> ；policy 观测中加入 <code>height_scan</code> ；动作空间保持 29DoF 关节位置控制；奖励配置和粗糙地形任务目标匹配
训练 checkpoint	20	提交有效 checkpoint，例如 <code>model_14000.pt</code> ；checkpoint 来自 <code>Unitree-G1-29dof-Velocity-Rough</code> 任务；能被 play 或 sim2sim 程序加载
运行视频	20	提交 IsaacSim play 或 MuJoCo sim2sim 运行视频；视频中机器人能在粗糙地形上保持运动，并能体现策略已经使用训练结果进行控制
简短报告	20	清楚说明从平地任务到粗糙地形任务修改了哪些观测、传感器、地形和奖励配置；说明 <code>height_scan</code> 的作用、扫描区域和观测维度；说明动作空间含义等

10. 常见问题 Q&A

Q1: 为什么训练过程发生中断，报错 `RuntimeError: normal expects all elements of std >= 0.0` 含义是策略网络输出的动作正态分布的标准差 (std/scale) 出现了负值？

A1: 这个报错说明训练数值不稳定，梯度爆炸导致网络参数崩坏，训练终止。在本次任务中很可能的原因是观测输入或奖励中 `height_scanner` 偶然输出 inf/NaN 引起的，可以在 `base_height_l2` 等使用到 `height_scanner` 传感器的奖励函数中加一些防护措施，避免异常值进入奖励和 loss。

Q2: 为什么我训练的机器人只能够学会原地站立，学不会行走？

A2: 可以适当增大 `track_lin_vel_xy` 等速度追踪奖励项的权重，直到机器人可以学会行走。此外，机器人走不好很大原因是奖励函数权重配置不合适，请耐心调整参数，多试几次直到获得满意的效果。如果不明白 unitree_rl_lab 各项奖励函数的作用，可以参看宇树G1机器人行走奖励函数详解：<https://www.bilibili.com/video/BV1tRmeB8EZA>

Q3: 为什么机器人在低洼处不断被重置？

A3: 因为终止项中有 `base height`，这一项的含义是：当机器人的 root（基座/躯干）在世界坐标系中的绝对高度低于 0.2 米时，episode 终止。在复杂地形下不建议开启该条件，因为机器人初始位置可能就在低洼处，root 高度可能一上来就低于 0.2m，直接终止导致训练无法正常进行。

Q4: 为什么机器人在所有地形上都被快速重置？

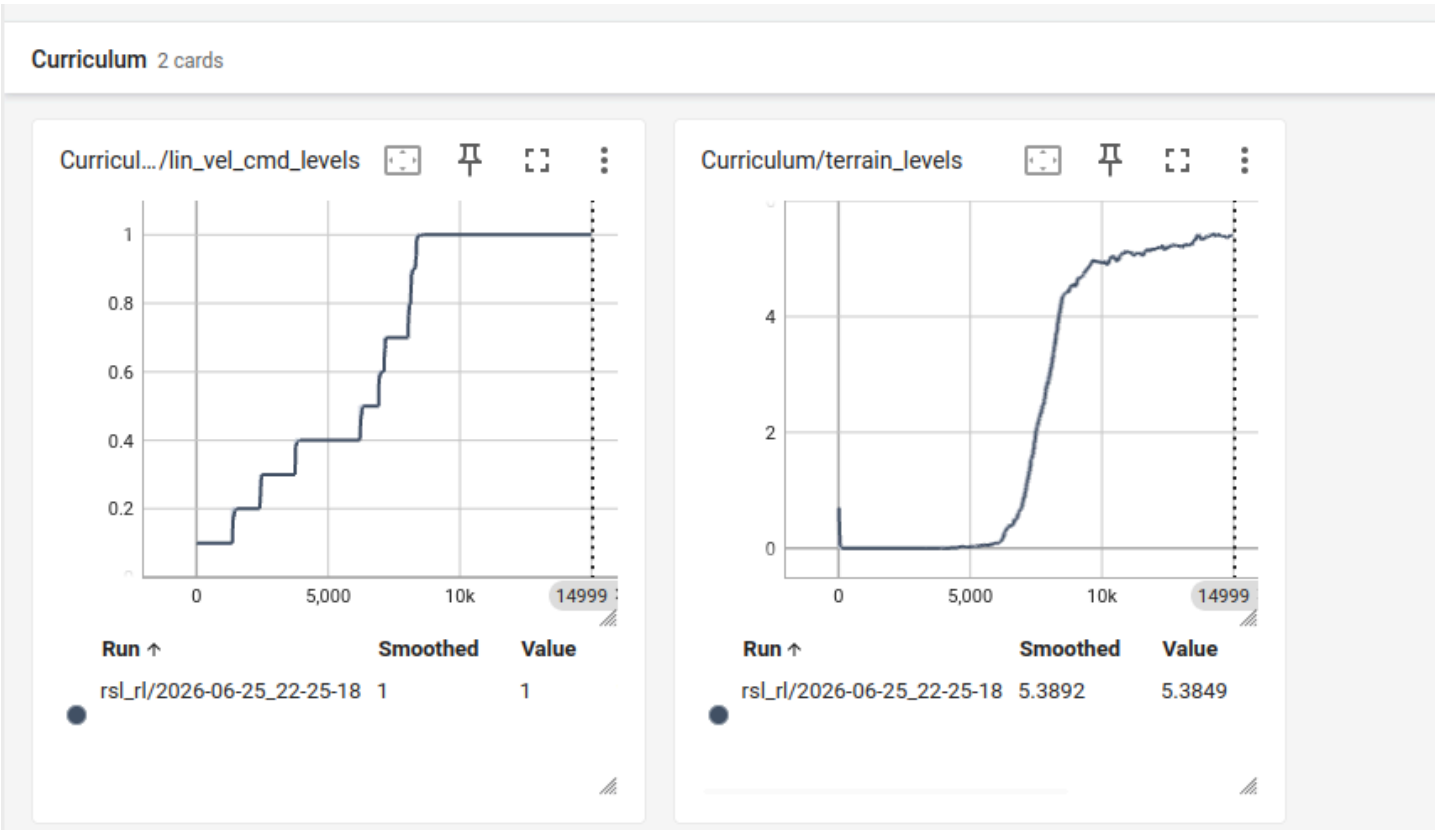
A4: 因为终止项 `illegal_contact = DoneTerm(func=mdp.illegal_contact, ...)`，可能会因为 reset 初期接触力瞬间达到阈值，导致机器人被不停重置，训练无法正常展开。而 `illegal_reset_contact` 如果阈值 `episode_length_threshold` 设置太小，也有可能导致机器人被不断重置，`Episode_Termination/illegal_reset_contact` 会接近1。

Q5: 如果等待训练到最大迭代次数再调试周期太长了，如何快速地调试奖励权重？

A5: 开多个轻量训练进程，`num_envs=1024`，每组权重分配短步数训练，`max_iterations=10000`，批量对比曲线，筛出 2-3 组优质权重再精细调优。

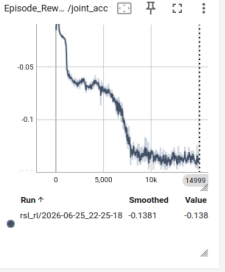
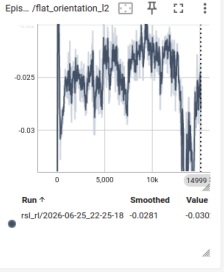
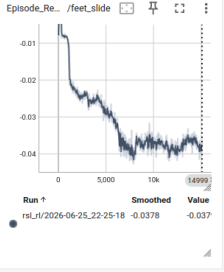
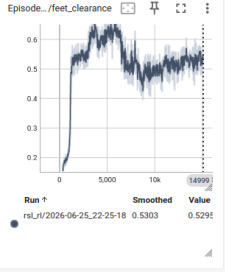
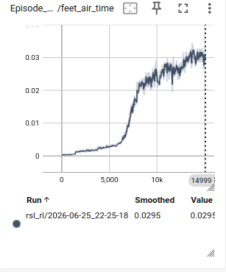
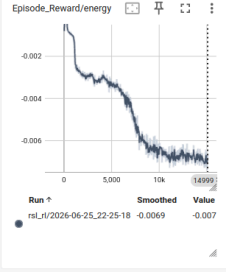
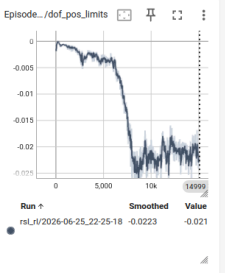
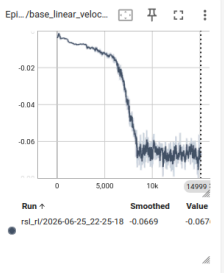
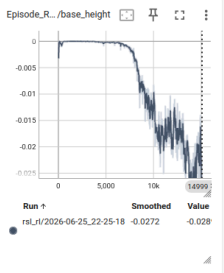
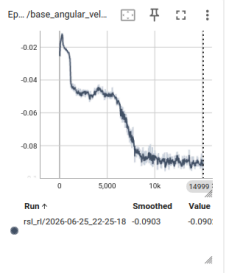
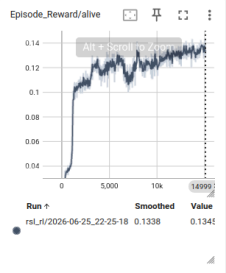
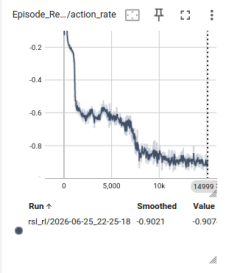
Q6: 正常的训练曲线是什么样子？

A6: 训练曲线如下，仅供参考。



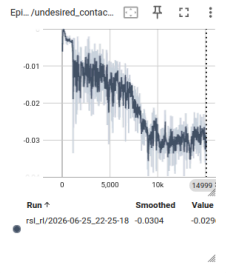
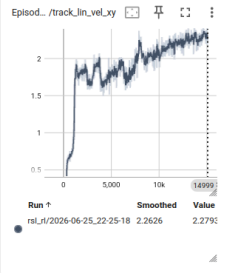
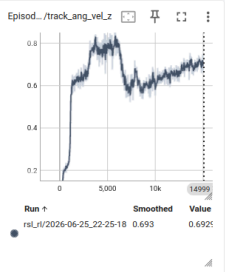
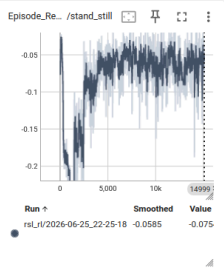
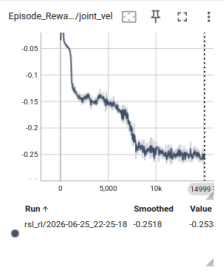
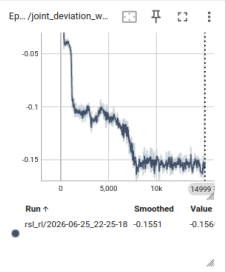
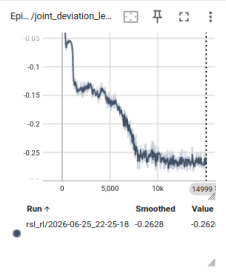
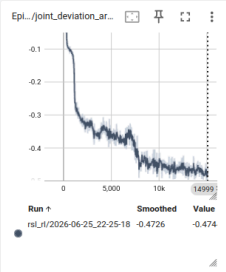
Previous

Next



Previous

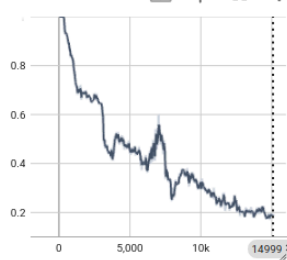
Next



Previous

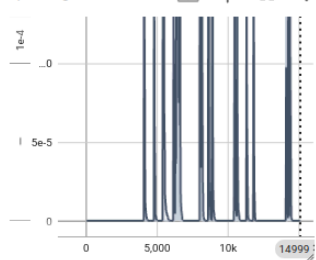
Episode_Termination 3 cards

Episod.../bad_orientation



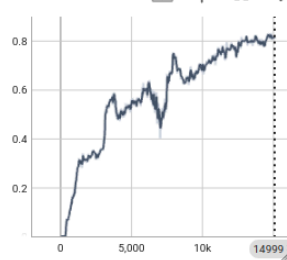
Run ↑ Smoothed Value
rsL_rl/2026-06-25_22-25-18 0.1872 0.191

Epl.../illegal_reset_cont...



Run ↑ Smoothed Value
rsL_rl/2026-06-25_22-25-18 0 0

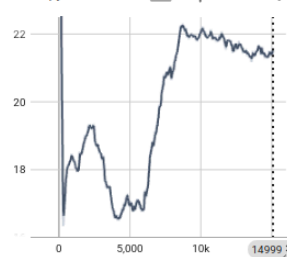
Episode_Ter... /time_out



Run ↑ Smoothed Value
rsL_rl/2026-06-25_22-25-18 0.8128 0.809

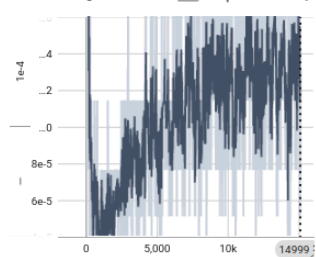
Loss 4 cards

Loss/entropy



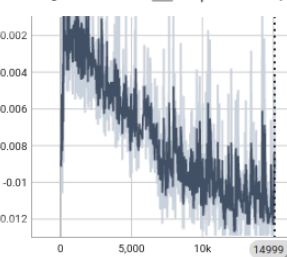
Run ↑ Smoothed Value
rsL_rl/2026-06-25_22-25-18 21.4821 21.518

Loss/learning_rate



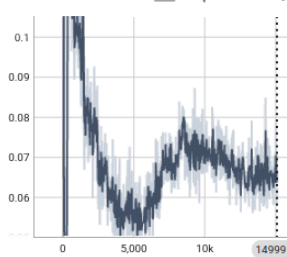
Run ↑ Smoothed Value
rsL_rl/2026-06-25_22-25-18 0.0001 0.0001

Loss/surrogate



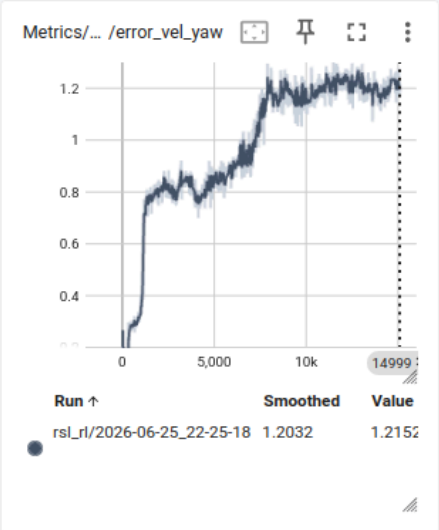
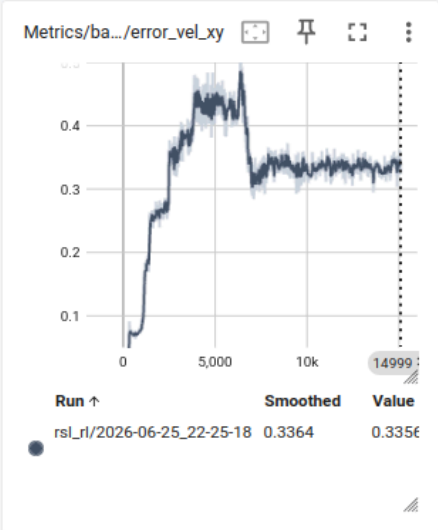
Run ↑ Smoothed Value
rsL_rl/2026-06-25_22-25-18 -0.0094 -0.010

Loss/value_function

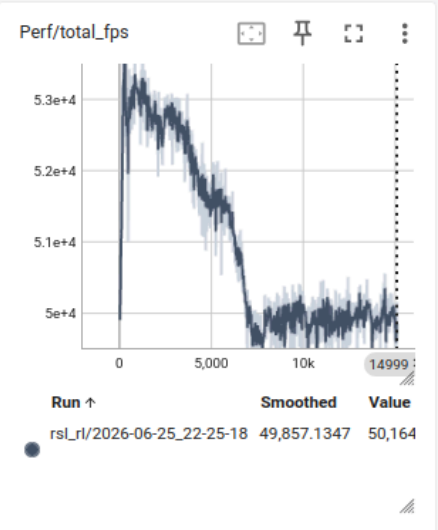
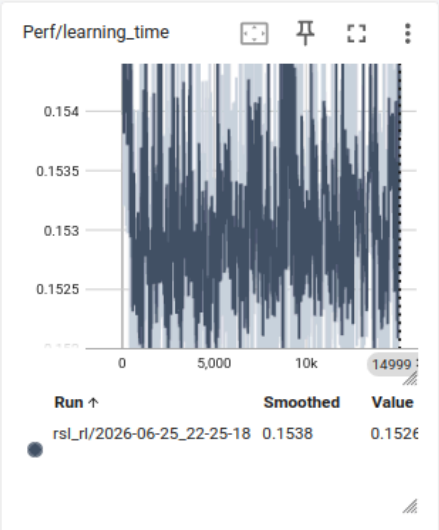
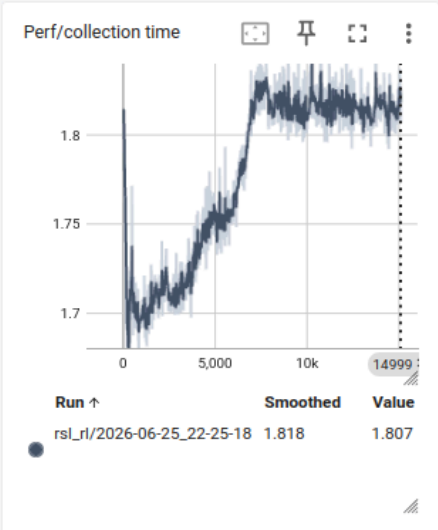


Run ↑ Smoothed Value
rsL_rl/2026-06-25_22-25-18 0.0633 0.0617

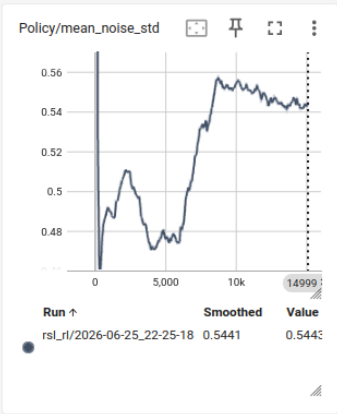
Metrics 2 cards



Perf 3 cards



Policy



Train 4 cards

